



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE ENSINO SUPERIOR DO SERIDÓ
BACHARELADO EM SISTEMAS DE INFORMAÇÃO.

JUAN VYCTOR SILVA GARCIA DE OLIVEIRA

TRABALHO AVALIATIVO DE ESTRUTURA DE DADOS

CAICÓ - RN
2026

JUAN VYCTOR SILVA GARCIA DE OLIVEIRA

TRABALHO AVALIATIVO DE ESTRUTURA DE DADOS:

Atividade do curso de graduação em Bacharelado em Sistemas de Informação, como parte dos requisitos para obtenção de nota na matéria de Estrutura de Dados.

Orientador(a): Dr. Arthur Emanuel Cassio da Silva e Souza.

Co-orientador(a): .

CAICÓ - RN

2026



Esta obra está licenciada com uma licença Creative Commons Atribuição 4.0 Internacional. Permite que outros distribuam, remixem, adaptem e desenvolvam seu trabalho, mesmo comercialmente, desde que creditem a você pela criação original. Link dessa licença: <<https://creativecommons.org/licenses/by/4.0/legalcode>>

RESUMO

Este trabalho apresenta uma análise experimental de estruturas de dados fundamentais, incluindo listas, filas e pilhas, implementadas com arrays dinâmicos e listas ligadas. O objetivo foi comparar o desempenho dessas estruturas a partir da medição de tempos de execução de operações básicas, como inserção, remoção e consulta, relacionando os resultados obtidos com as respectivas complexidades assintóticas (Big-O). Para isso, foram desenvolvidos testes automatizados que executam cada operação com diferentes tamanhos de entrada, permitindo a geração de gráficos comparativos entre as implementações baseadas em array e em lista ligada. Os resultados evidenciam diferenças significativas de desempenho entre as abordagens, especialmente em operações que envolvem deslocamento de elementos ou percorrimto de nós, destacando o impacto da escolha da estrutura de dados na eficiência computacional.

Palavras-chave: Estruturas de Dados; Lista; Fila; Pilha; Lista Ligada; Array Dinâmico; Complexidade de Algoritmos; Big-O; Análise Experimental

ABSTRACT

This work presents an experimental analysis of fundamental data structures, including lists, queues, and stacks, implemented using dynamic arrays and linked lists. The main objective was to compare the performance of these structures by measuring the execution time of basic operations such as insertion, deletion, and access, relating the obtained results to their asymptotic time complexities (Big-O notation). To achieve this, automated tests were developed to execute each operation with different input sizes, enabling the generation of comparative graphs between array-based and linked-list-based implementations. The results highlight significant performance differences between the approaches, especially in operations involving element shifting or node traversal, emphasizing the impact of data structure selection on computational efficiency.

Keywords: Data Structures; List; Queue; Stack; Linked List; Dynamic Array; Algorithm Complexity; Big-O; Experimental Analysis

SUMÁRIO

1	INTRODUÇÃO	5
2	CONFIGURAÇÕES DE AMBIENTE	5
3	EXECUÇÃO	5
3.1	Gráficos individuais	5
3.1.1	Fila - Array Dinâmico	6
3.1.2	Fila - Lista Ligada	7
3.1.3	Pilha - Array Dinâmico	8
3.1.4	Pilha - Lista Ligada	9
3.1.5	Lista - Array Dinâmico	10
3.1.6	Lista - Lista Ligada	11
3.2	Gráficos comparativos	12
3.2.1	Fila	12
3.2.2	Pilha	13
3.2.3	Lista	14

1 INTRODUÇÃO

O trabalho a seguir apresenta uma análise empírica de estruturas de dados fundamentais, especificamente listas, filas e pilhas, implementadas com Arrays dinâmicos e Listas ligadas. O objetivo é comparar a performance dessas implementações por meio da medição do tempo de execução das principais, como inserção e remoção, relacionando os resultados com suas respectivas complexidades.

A metodologia utilizada foi a execução automática de testes com diferentes tamanhos de entrada, que possibilitou a observação do tempo de execução e como o mesmo muda conforme o crescimento dos dados. Os resultados foram separados em arquivos e utilizados para a construção de gráficos individuais e comparativos entre cada tipo de estrutura.

Por meio dessa análise, busca-se evidenciar o impacto da escolha da estrutura de dados no desempenho computacional, destacando as diferenças entre abordagens baseadas em Arrays Dinâmicos e Listas Ligadas, e reforçando a importância da análise de complexidade na avaliação de algoritmos e estruturas de dados.

2 CONFIGURAÇÕES DE AMBIENTE

A máquina utilizada para os experimentos foi um MacBook Pro 2017, com um processador Intel Dual Core i5 2,3GHz, placa gráfica Intel Iris Plus Graphics 640 1536 MB, uma memória RAM 8 GB 2133 MHz LPDDR3, e um sistema operacional macOS Ventura versão 13.7.8. Em relação ao Python, a versão instalada na máquina é a 3.12.2 e a biblioteca matplotlib está em sua versão 3.10.9.

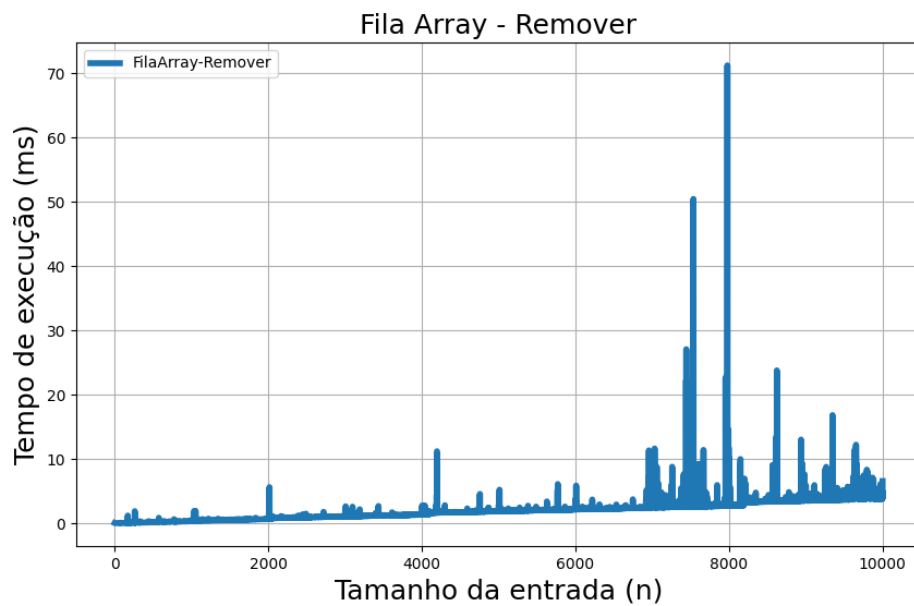
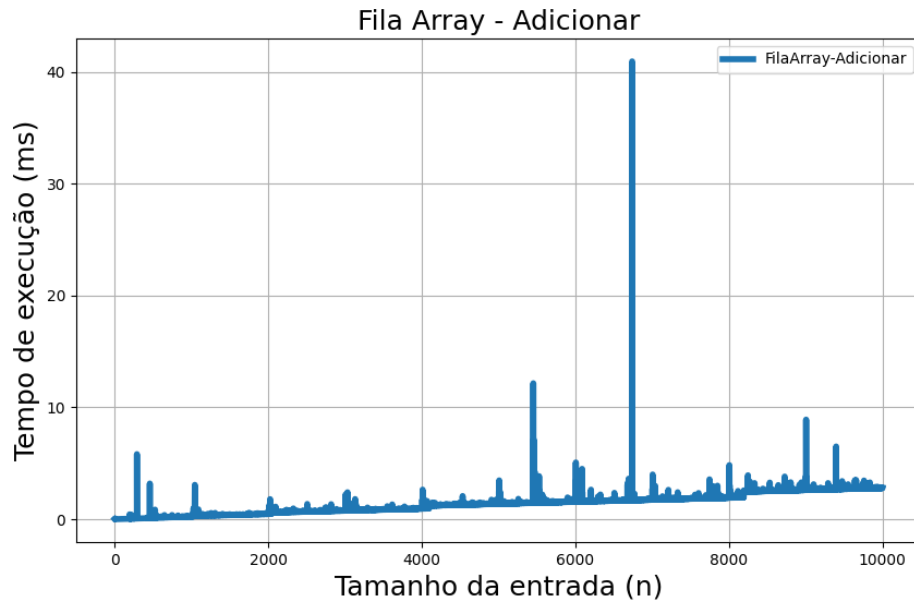
3 EXECUÇÃO

3.1 Gráficos individuais

Inicialmente foi feito o experimento com cada tipo de estrutura, utilizando seus métodos de inserção e remoção. Por questões de conveniência e facilidade de nomeação, os gráficos estão descritos apenas como "Adição" ou "Remoção", enquanto na verdade, estruturas diferentes possuem métodos distintos, como `append()` e `insert()` para inserções, e `remove()` e `pop()` para remoções.

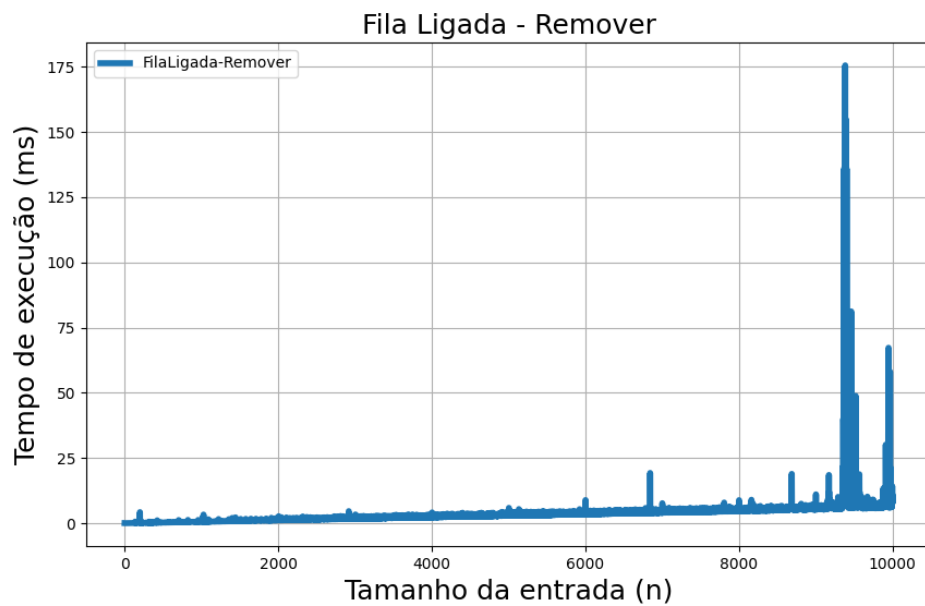
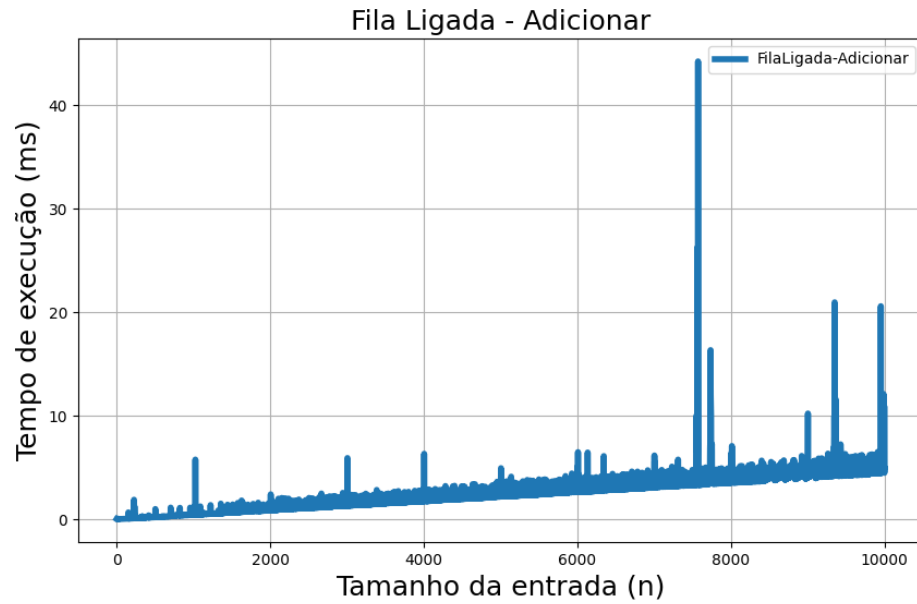
A seguir, cada par de gráfico receberá uma breve explicação de complexidade e funcionamento, com um gráfico relacionado. Devido à forma que o algoritmo de execução foi construído, alguns gráficos podem ficar um pouco diferente do que a complexidade indica.

3.1.1 Fila - Array Dinâmico



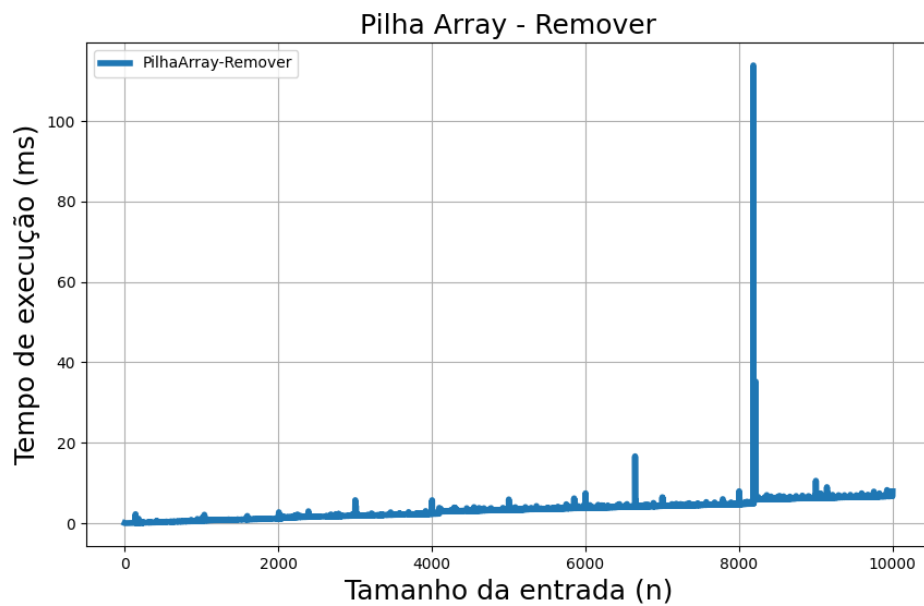
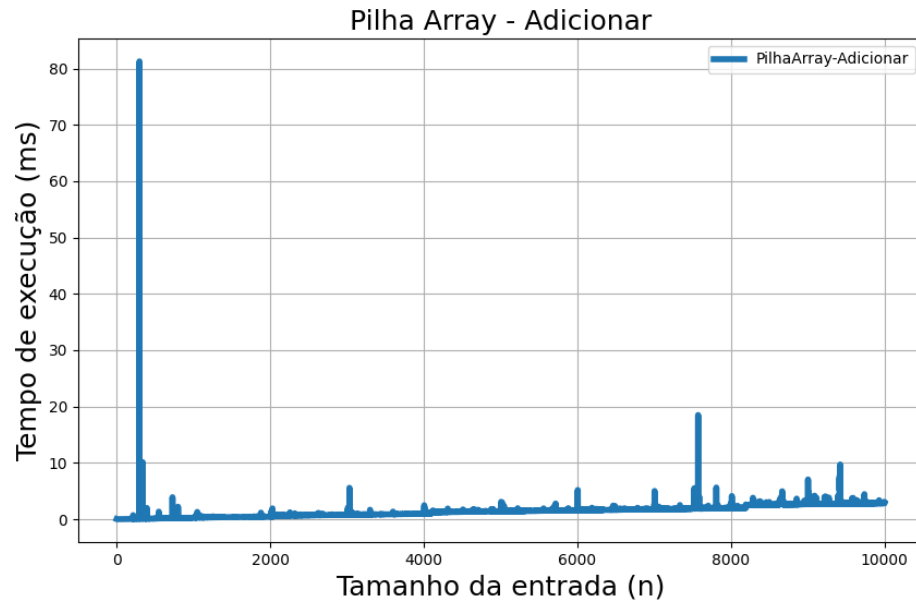
Na fila com Array, a adição tem complexidade $O(1)$, já que é apenas adicionado um elemento, mas sua remoção é $O(N)$, já que é necessário o reajuste das posições anteriores.

3.1.2 Fila - Lista Ligada



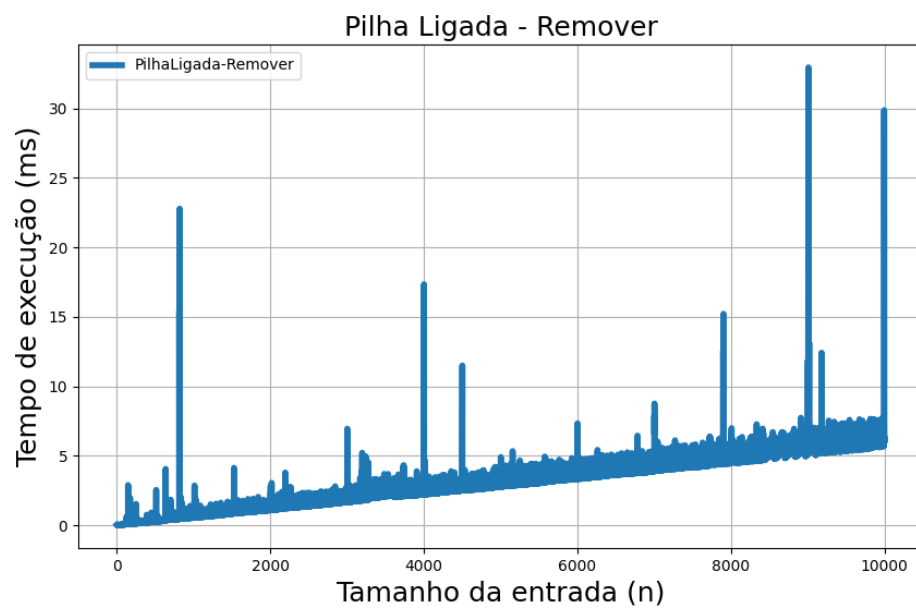
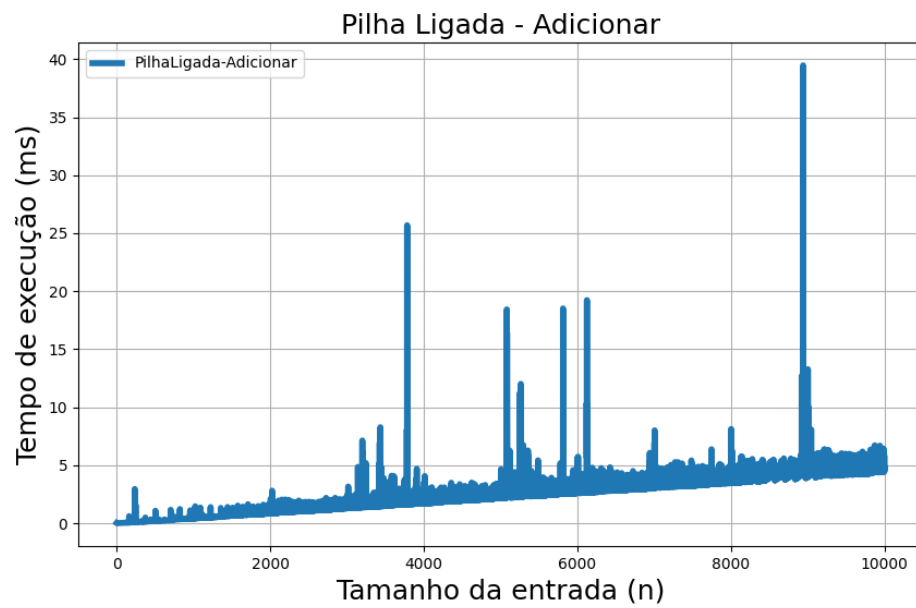
Com lista ligada, tanto adição e remoção de itens tem complexidade $O(1)$, já que o nó tem referência ao fim, onde é adicionado um elemento, e a remoção apenas tira o primeiro índice do nó.

3.1.3 Pilha - Array Dinâmico



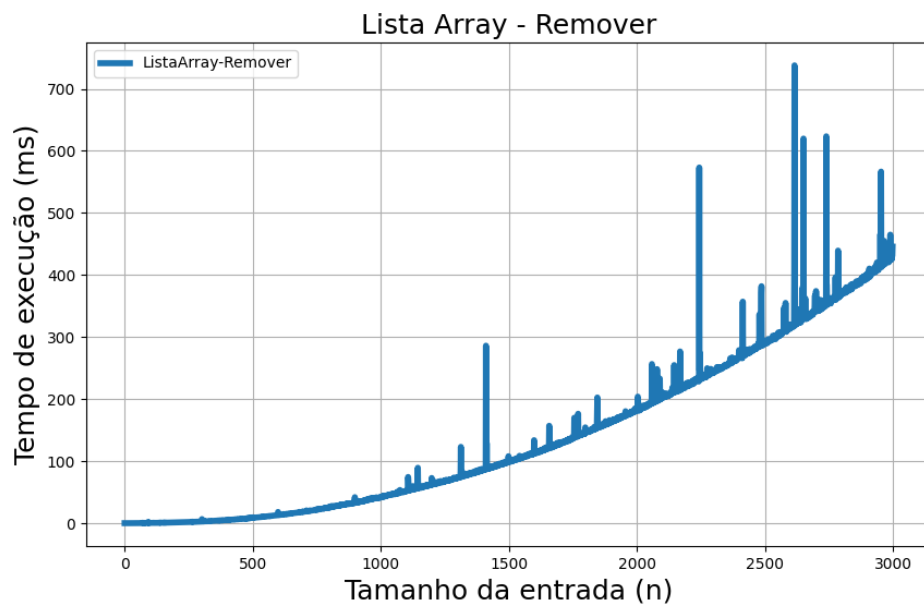
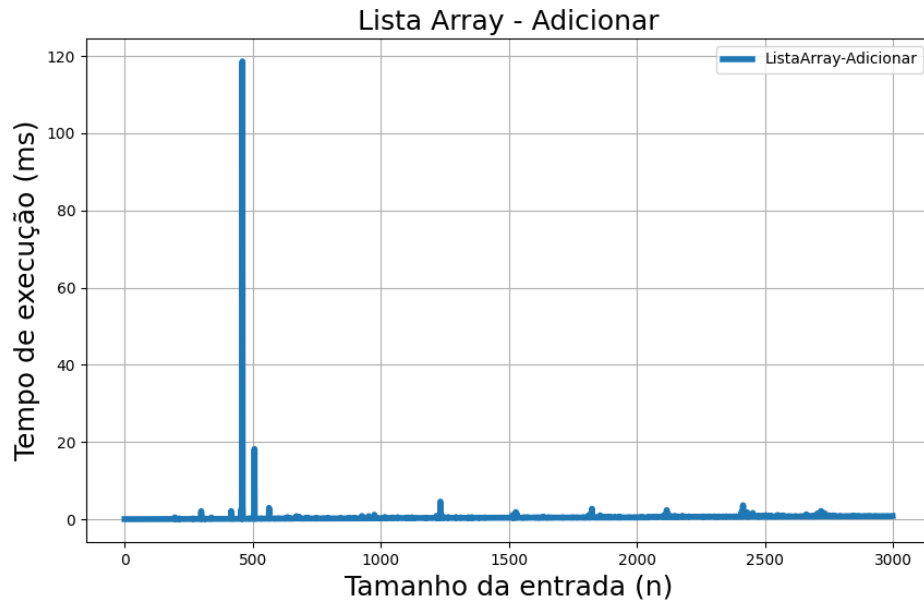
Na pilha com Array, similar às filas com listas, os casos são $O(1)$, já que o novo item sempre vem ao fim, e a remoção sempre retira o mais recente, sem maiores alterações.

3.1.4 Pilha - Lista Ligada



A mesma lógica se aplica em situações que fazem o uso da lista ligada, se mantendo $O(1)$, mesmo que as operações sejam um pouco mais custosas.

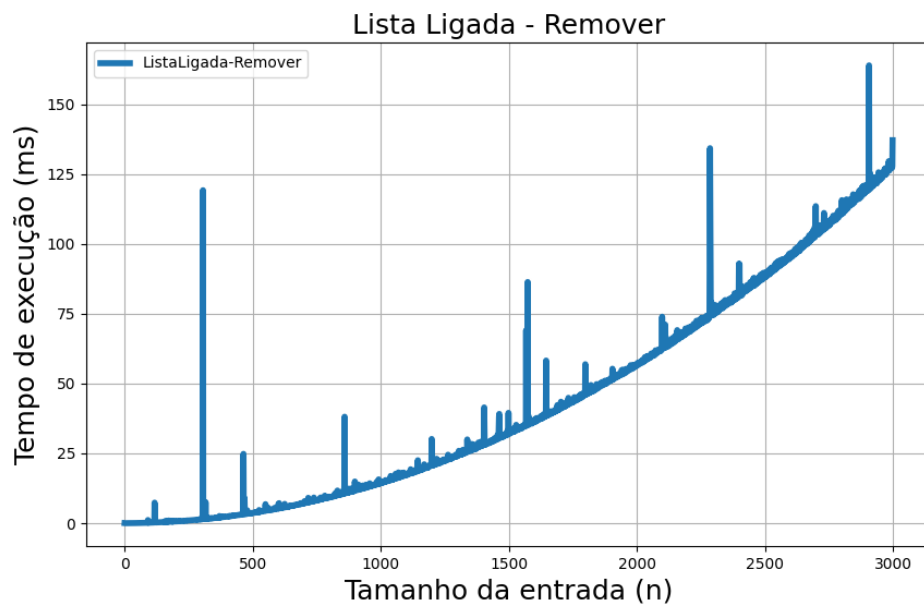
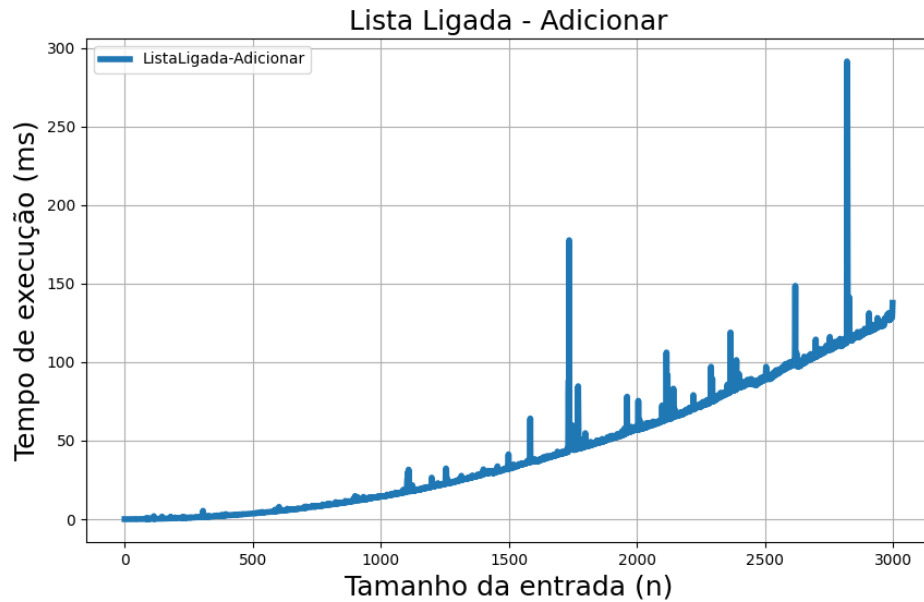
3.1.5 Lista - Array Dinâmico



A inserção de um novo elemento é sempre $O(1)$, por ser inserido ao final, mas a retirada de um item é $O(N)$, já que o que vier após o elemento removido precisa ser alterado.

No caso da remoção, o algoritmo de execução acaba tendo um caso de "N sendo chamado N vezes", que torna o visual erroneamente $O(N^2)$.

3.1.6 Lista - Lista Ligada



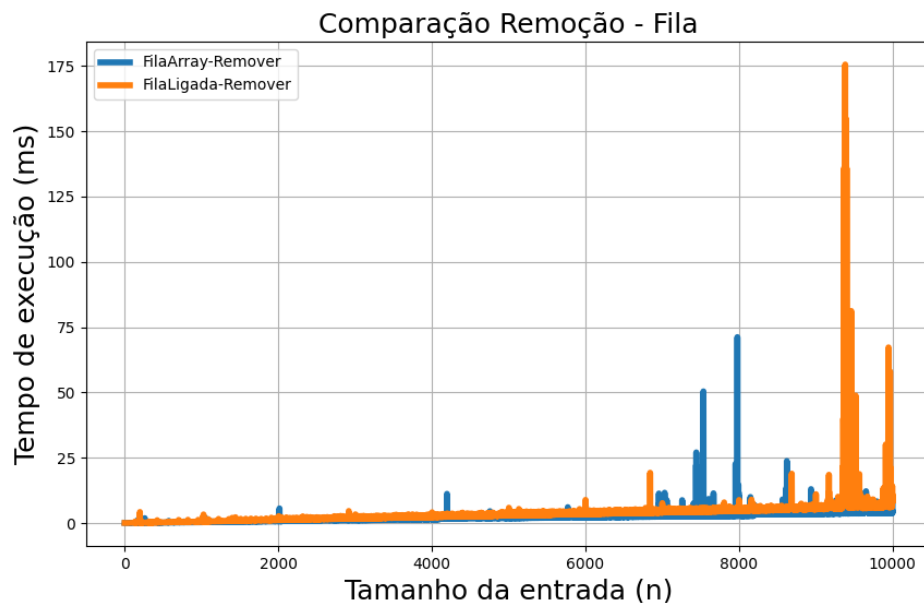
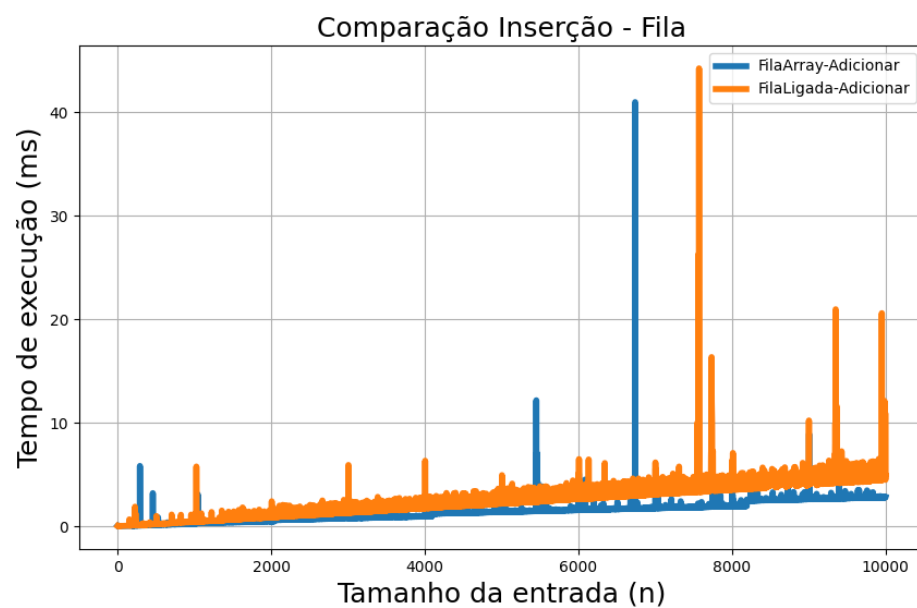
Por fim, com lista ligada, as duas situações são $O(N)$, já que, partindo do item inicial da lista, é necessário se locomover até a posição desejada para a operação, e então fazer a alteração desejada.

Nos dois casos, o algoritmo de execução acaba tendo um caso de "N sendo chamado N vezes", que torna o visual erroneamente $O(N^2)$.

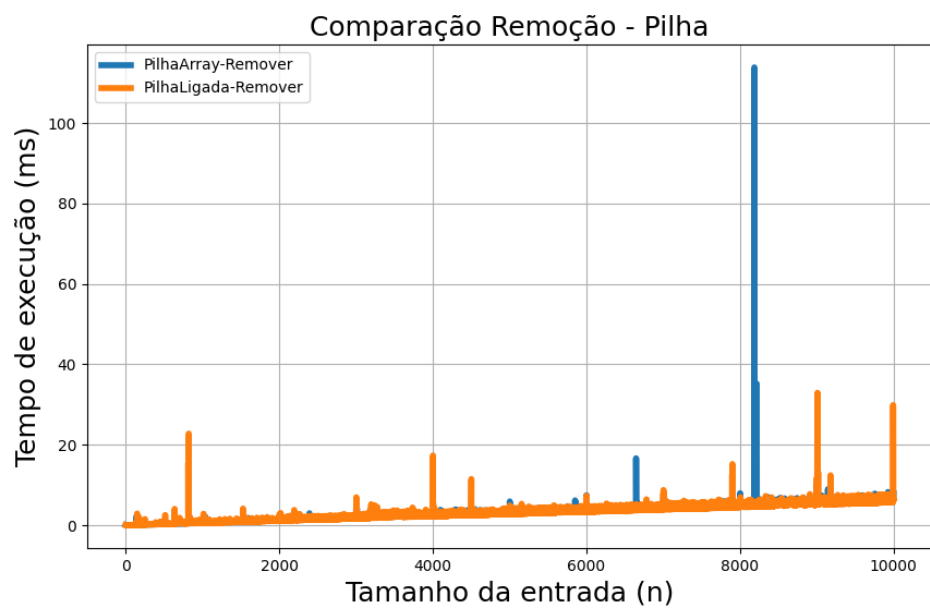
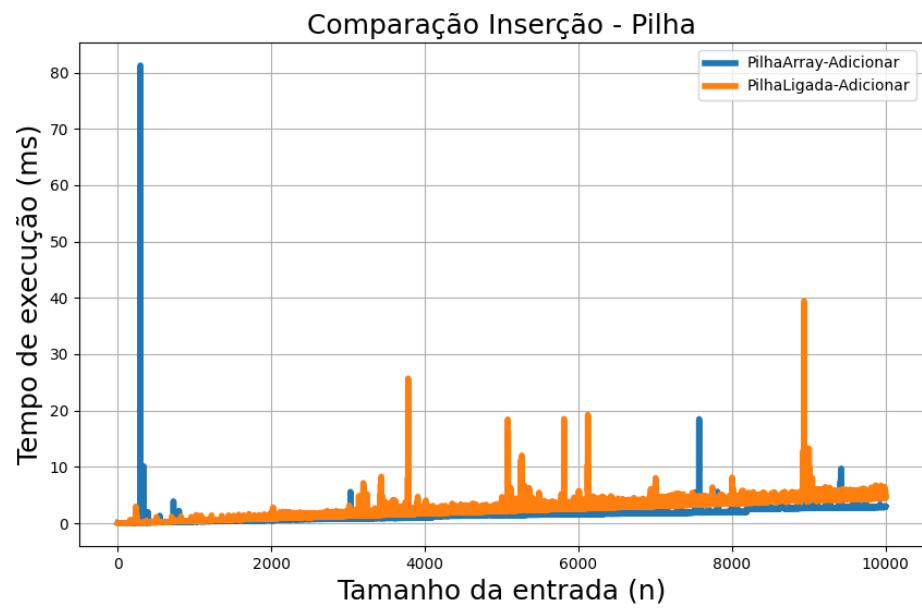
3.2 Gráficos comparativos

Aqui serão expostos gráficos comparando cada uma das operações ao utilizarmos os diferentes tipos de armazenagem, Arrays Dinâmicos e Lista Ligada. Em cada um deles está colorido de forma distinta, e como foi explicado individualmente a cima, podemos ver como cada situação tem tempos distintos para cada tipo de armazenagem de dados.

3.2.1 Fila



3.2.2 Pilha



3.2.3 Lista

